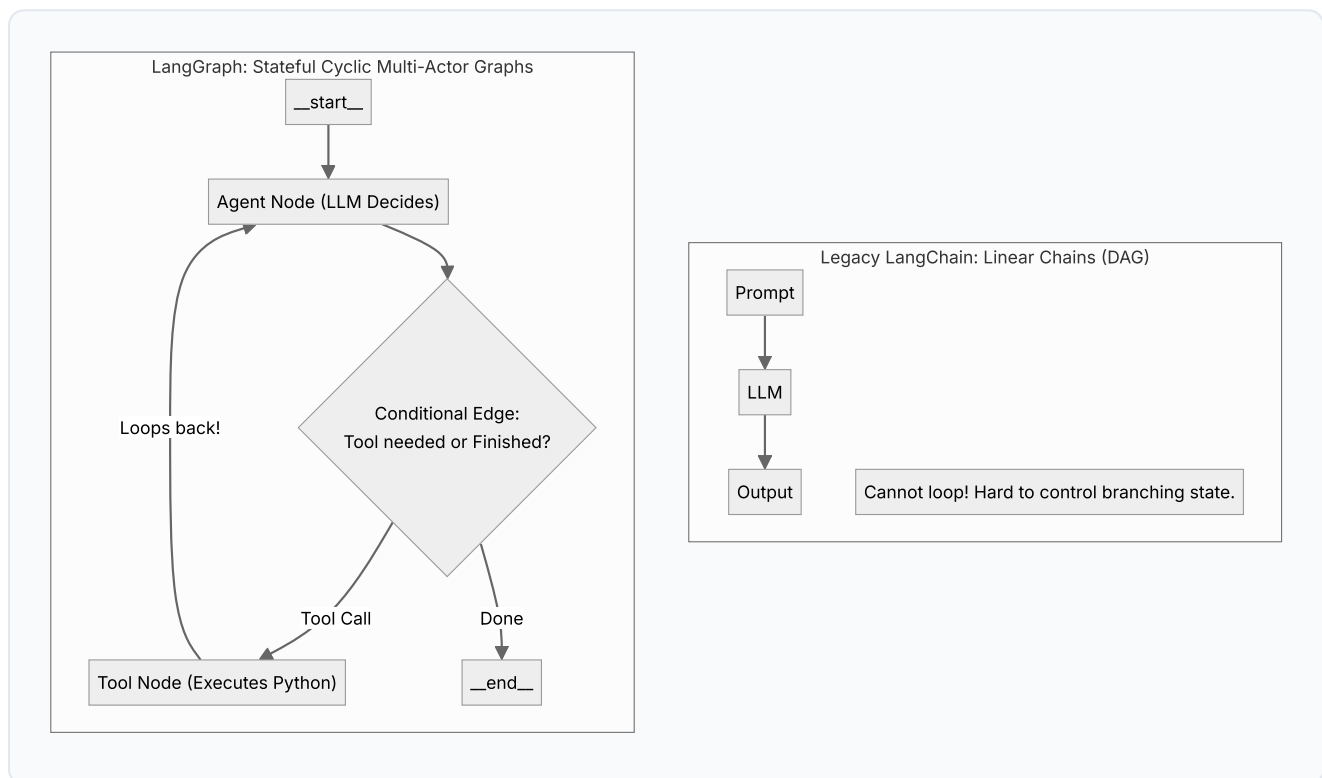


Agentic AI Chapter 5: LangChain & LangGraph Stateful Workflow Mastery

Core Learning Objective: Master modern stateful agent systems using LangGraph. Understand cyclic graphs, typed state schemas, conditional branching edges, persistence checkpoints, and Human-in-the-Loop approval workflows.

1. Why LangGraph Replaced Linear Chains

Legacy frameworks (LangChain chains, LCEL) were designed as **Directed Acyclic Graphs (DAGs)**. But real agents are **inherently cyclic**: they try an action, inspect output, loop back, retry, ask clarifications, and refine results.



2. Core LangGraph Primitives

1. **State:** A centralized typed dictionary (`TypedDict`) shared across all nodes in the graph. Nodes return partial updates that merge into the state.

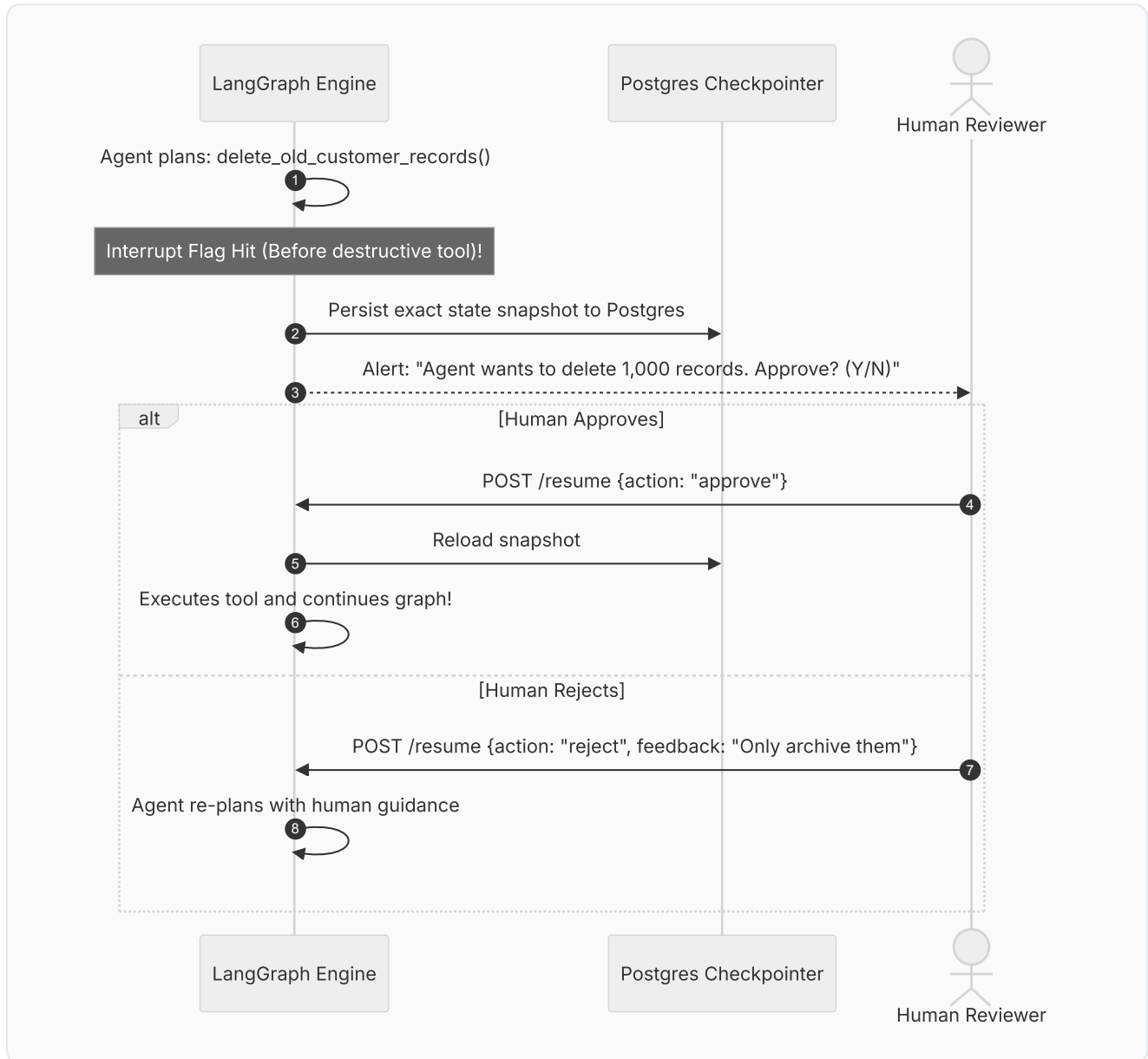
2. **Nodes:** Plain Python functions that receive the current `State`, perform computation (call an LLM or execute a tool), and return updated keys.
 3. **Edges:** Direct transitions from one node to another.
 4. **Conditional Edges:** Dynamic routers that inspect the state and determine the next destination node.
-

3. LangGraph-Style Agent (Pure-Python Emulation of the Model)



4. Human-in-the-Loop & Time-Travel Checkpointing

In production, agents executing destructive actions (e.g. modifying financial ledgers, sending mass emails, dropping tables) must be paused for **Human Approval**:



Time-travel checkpointing allows developers to inspect every historical node execution, modify past state variables, and branch alternative execution paths during debugging.

Verified Worked Example: Real LangGraph with Human-in-the-Loop

The section above emulates the LangGraph execution model in plain Python to teach the ideas. The real API is exercised in `examples/ex02_langgraph_hitl.py`, verified on `langgraph=1.2.13`: a typed `State` with an append reducer (`Annotated[list, operator.add]`), nodes returning partial updates, a conditional edge, an `InMemorySaver` checkpointer, and an `interrupt()` that pauses for approval and resumes with `Command(resume= ...)` on the same `thread_id`. Two behaviours worth testing yourself: (1) the side-effecting node runs **only after** approval, and (2) rejecting leaves `executed` out of the log.

Version note. LangGraph's API evolves quickly (for example `MemorySaver` became `InMemorySaver`, and the human-in-the-loop primitives changed from static breakpoints to `interrupt` / `Command`). Pin the version and re-run the examples when upgrading.

Pinned for the verified examples: `langgraph=1.2.13`, `mcp=2.3.0`, `pytest=9.1.1` (see `examples/requirements.txt`). All examples run offline with a scripted fake model: `cd examples && pip install -r requirements.txt && pytest -q`.

5. Runnable Model: What a Graph Runtime Actually Does

This is a **teaching emulation** of the ideas in LangGraph (state with reducers, conditional edges, checkpoint and resume), small enough to read in one sitting. The real library with pinned versions is in `examples/ex02_langgraph_hitl.py`; use it in projects.

What to take from it

1. A graph agent is **state plus nodes plus edges**. Nodes return *updates*; **reducers** decide how updates combine (append messages, overwrite a flag).
2. **Conditional edges** are where the agent decides what to do next; cycles are legal, so you must bound them.
3. **Checkpointing** (saving the state at each step) is what makes human approval, retries and crash recovery possible: resume from the saved state instead of starting again.
4. Choose a graph framework when you need **branching, loops, persistence or human approval**; for a straight sequence of calls, plain functions are clearer.

Further Reading

- [LangGraph documentation](#)
- [LangGraph persistence](#)
- [LangGraph graph API](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

- 1. What does a reducer do in LangGraph state?
- 2. What does a checkpointer enable?

- ▶ **3.** What does `interrupt()` plus `Command(resume=...)` do?
- ▶ **4.** Why re-run examples when upgrading LangGraph?